INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Ingeniería en Inteligencia Artificial

Unidad de aprendizaje: Análisis y Diseño de Algoritmos

Grupo: 3BV1

Práctica 3

Algoritmos Voraces y Codificación Huffman

Alumna: Molina Zavaleta Angie Lalaine

Fecha: 10 de noviembre de 2026

Código fuente: *huffman.cpp* (incluido en el Apéndice A y en el repositorio).

Fase 1. Teoría

Mensaje: $\mathcal{M}$ = **"ABRACADABRA_ALGORITMO_AVANZADO_ESCOM"**, con n = 36 caracteres y 17 símbolos distintos.

1.A Tabla de frecuencias

Símbolo	Frecuencia absoluta f_i	Frecuencia relativa $p_i = f_i/36$
A	9	9/36 = 0.2500
O	4	4/36 = 0.1111
R	3	3/36 = 0.0833
_ (guion bajo)	3	3/36 = 0.0833
B	2	2/36 = 0.0556
C	2	2/36 = 0.0556
D	2	2/36 = 0.0556
M	2	2/36 = 0.0556
E	1	1/36 = 0.0278
G	1	1/36 = 0.0278
I	1	1/36 = 0.0278
L	1	1/36 = 0.0278
N	1	1/36 = 0.0278
S	1	1/36 = 0.0278
T	1	1/36 = 0.0278
V	1	1/36 = 0.0278
Z	1	1/36 = 0.0278
Total	**36**	**1.0000**

1.B Entropía empírica de Shannon

$$H(\mathcal{M}) = -\Sigma\, p_i \log_2 p_i = \log_2(36) - (1/36)\, \Sigma\, f_i \log_2 f_i$$

$\Sigma\, f_i \log_2 f_i = 9\cdot\log_2 9 + 4\cdot\log_2 4 + 2\cdot(3\cdot\log_2 3) + 4\cdot(2\cdot\log_2 2) = 28.529 + 8 + 9.510 + 8 = 54.039$ (los símbolos con f = 1 aportan 0).

$H(\mathcal{M}) = 5.1699 - 54.039/36 = 5.1699 - 1.5011 =$ **3.6688 bits/símbolo** (≈ 132.1 bits en total para el mensaje).

1.C Construcción del árbol de Huffman (estados del bosque)

En cada paso se extraen los dos nodos de menor peso y se fusionan (el primero extraído es el hijo izquierdo, rama 0; el segundo, el derecho, rama 1). En los empates se prefieren las hojas sobre los nodos internos. Notación: n_k = nodo interno creado en el paso k.

Paso	Nodos fusionados	**Resultado**
Inicial	A9, O4, R3, _3, B2, C2, D2, M2, L1, G1, I1, T1, V1, N1, Z1, E1, S1	—
1	L1 + G1	n1 = 2
2	I1 + T1	n2 = 2
3	V1 + N1	n3 = 2
4	Z1 + E1	n4 = 2
5	S1 + B2	n5 = 3
6	C2 + D2	n6 = 4

Paso	Nodos fusionados	Resultado
7	M2 + n1(2)	n7 = 4
8	n2(2) + n3(2)	n8 = 4
9	n4(2) + R3	n9 = 5
10	_3 + n5(3)	n10 = 6
11	O4 + n6(4)	n11 = 8
12	n7(4) + n8(4)	n12 = 8
13	n9(5) + n10(6)	n13 = 11
14	n11(8) + n12(8)	n14 = 16
15	A9 + n13(11)	n15 = 20
16	n14(16) + n15(20)	raíz = 36

Estados intermedios del bosque (pesos ordenados):

Momento	Bosque
Tras el paso 4	S1, B2, C2, D2, M2, n1:2, n2:2, n3:2, n4:2, R3, _3, O4, A9
Tras el paso 8	n4:2, R3, _3, n5:3, O4, n6:4, n7:4, n8:4, A9
Tras el paso 12	n9:5, n10:6, n11:8, n12:8, A9
Tras el paso 15	n14:16, n15:20
Tras el paso 16	raíz:36 (un solo árbol)

Estructura del árbol resultante (0 = izquierda, 1 = derecha):

```
raíz(36)
├─0─ n14(16)
│      ├─0─ n11(8) ── 0: O        1: n6(4) ── 0: C   1: D
│      └─1─ n12(8)
│            ├─0─ n7(4) ── 0: M   1: n1(2) ── 0: L   1: G
│            └─1─ n8(4)
│                  ├─0─ n2(2) ── 0: I   1: T
│                  └─1─ n3(2) ── 0: V   1: N
└─1─ n15(20)
       ├─0─ A
       └─1─ n13(11)
             ├─0─ n9(5) ── 0: n4(2) ── 0: Z   1: E     1: R
             └─1─ n10(6) ── 0: _      1: n5(3) ── 0: S   1: B
```

1.D Tabla de códigos, longitud promedio y desigualdad de Shannon

Símbolo	f_i	Código	$\lvert c_i \rvert$	$f_i \cdot \lvert c_i \rvert$
A	9	10	2	18
O	4	000	3	12
R	3	1101	4	12
_	3	1110	4	12
B	2	11111	5	10
C	2	0010	4	8
D	2	0011	4	8
M	2	0100	4	8
E	1	11001	5	5
G	1	01011	5	5
I	1	01100	5	5
L	1	01010	5	5
N	1	01111	5	5

Símbolo	f_i	Código	$\lvert c_i \rvert$	$f_i \cdot \lvert c_i \rvert$
S	1	11110	5	5
T	1	01101	5	5
V	1	01110	5	5
Z	1	11000	5	5
Total	**36**			**133**

$L = \Sigma\, p_i \cdot |c_i| = 133/36 =$ **3.6944 bits/símbolo**.

Verificación de Kraft: $\Sigma\, 2^{-|c_i|} = 1 \cdot ¼ + 1 \cdot ⅛ + 5 \cdot (1/16) + 10 \cdot (1/32) = 1$ (código completo y libre de prefijos).

Desigualdad de Shannon: $H(\mathcal{M}) \le L < H(\mathcal{M}) + 1 \rightarrow$ **3.6688 ≤ 3.6944 < 4.6688**. Se cumple; la redundancia es $L - H = 0.0256$ bits/símbolo.

Comparación: ASCII usaría 288 bits, un código de longitud fija de 5 bits usaría 180 bits y Huffman usa 133 bits (≈ 54 % menos que ASCII).

2. Protocolo de serialización

El descompresor no conoce el alfabeto a priori, por lo que el archivo debe contener la información para reconstruir el árbol.

Opción	**Contenido**	Sobrecarga (k = 17 hojas)	Sobrecarga general
A. Tabla de frecuencias	256 enteros de 4 bytes	256·4 = 1024 bytes	1024 bytes (constante)
B. Árbol en preorden	1 bit por nodo (0 = interno, 1 = hoja) + 8 bits por hoja	(k−1) + k·9 = 169 bits ≈ 22 bytes	10k − 1 bits (≤ 2559 bits ≈ 320 bytes con k = 256)

Con k hojas, el árbol tiene k − 1 nodos internos, es decir, 2k − 1 nodos en total (cada uno aporta 1 bit) y k hojas aportan 8 bits adicionales: (2k − 1) + 8k = 10k − 1 bits. Para este mensaje, los datos codificados ocupan ⌈133/8⌉ = 17 bytes, de modo que con la Opción A el archivo sería de 1041 bytes y con la Opción B de 39 bytes. Ambos superan los 36 bytes del original: en mensajes muy cortos la cabecera domina. Se elige la **Opción B**.

Formato implementado: ["HUF1" (4 B)] [N = número de caracteres originales, uint64] [árbol en preorden] [datos codificados] [relleno de ceros hasta completar el byte].

Fase 2. Uso de la IA

2.1 Auditoría del prompt ingenuo (cadenas de texto binarias)

Prompt ingenuo: «Escribe una función en C++ que recorra el archivo de entrada, sustituya cada carácter por su código de Huffman devuelto como std::string (ejemplo: "01011"), concatene todas las cadenas en un gran string de texto y luego lo guarde en disco.»

A. Por qué no comprime

Un carácter '0' o '1' en un std::string ocupa 1 byte (8 bits) para representar 1 bit de información. Si el flujo codificado tiene N bits útiles, el string ocupa N bytes, es decir, 8 veces más que el flujo real (N/8 bytes). Con el mensaje de la práctica: 133 bits → 133 bytes de texto en lugar de 17 bytes empaquetados (7.8 veces más), y 133 bytes es incluso mayor que los 36 bytes originales (3.7 veces). Guardar ese string en disco no comprime nada: se genera un archivo de texto con ceros y unos que ocupa más que la entrada. La información de la codificación se conserva, pero el formato de almacenamiento anula toda la ganancia.

B. Desperdicio de memoria y sobrecosto temporal

Sea n el número de caracteres de entrada y $\bar{\ell}$ la longitud promedio de código, de modo que $N = n \cdot \bar{\ell}$.

• **Espacio:** el string usa N bytes frente a N/8 necesarios, desperdiciando 7N/8 bytes (87.5 %). Además, cada código intermedio devuelto como std::string crea un objeto temporal con su propio encabezado (≈ 32 bytes en libstdc++), lo que agrega memoria transitoria.

• **Tiempo (concatenación ingenua s = s + code):** en el paso i se copian $i \cdot \bar{\ell}$ bytes, de modo que $T(n) = \Sigma_{i=1..n}\, i \cdot \bar{\ell} = \bar{\ell} \cdot n(n+1)/2 = \mathbf{O(n^2)}$.

• **Con s += code y crecimiento geométrico:** las realocaciones copian a lo sumo N + N/2 + N/4 + … ≤ 2N bytes, así que el costo amortizado es O(N) = O(n), pero el pico de memoria llega a ≈ 3N bytes, porque el buffer viejo y el nuevo coexisten durante la copia. Sin reserve() y con crecimiento aditivo constante, vuelve a ser $O(n^2)$.

C. Contra-prompt

«Rechazo la solución con strings de '0' y '1'. Reescribe el compresor con una clase BitWriter y una BitReader que empaqueten los bits en un std::vector<uint8_t> usando desplazamientos (<<, >>) y máscaras (&, |). BitWriter debe tener put_bit(), put_bits(valor, n) y flush(). La tabla de códigos debe guardarse como vectores de bits, no como strings. El archivo debe llevar cabecera con el tamaño original como uint64 y el árbol serializado en preorden. Muestra el código completo.»

La clase resultante (Apéndice A) acumula los bits en un byte con `cur = (cur << 1) | (b & 1)`, lo escribe al completar 8 bits y, en `flush()`, desplaza `cur << (8 − nbits)` para alinear a la izquierda. BitReader extrae el bit con `(buf[pos >> 3] >> (7 − (pos & 7))) & 1`.

2.2 El bug del último byte (The Padding Trap)

Si el flujo tiene 11 bits (10110011 010), el último byte se guarda como 01000000: los 5 ceros finales son relleno, pero el decodificador no puede distinguirlos de bits reales.

A y B. Evaluación del decodificador ingenuo

Un decodificador que lee «hasta agotar los bits» (función `decompress_naive` del código) interpreta el relleno como datos. Si los ceros finales forman el prefijo de un código válido, se emite un carácter espurio. En el árbol de este mensaje, el código de 'O' es 000: con 3 bits de relleno el decodificador produciría una 'O' de más. El bug es intermitente: depende de cuántos bits sobren y de qué códigos

empiecen con ceros. En la prueba con el mensaje (2 bits de relleno, menos que cualquier código que empieza con 0) la salida fue correcta, pero el caso es frágil y falla con otros tamaños de entrada.

C. Corrección

Se implementó la segunda técnica de la práctica: registrar en la cabecera el número total de caracteres originales N como entero de 64 bits. El decodificador se detiene exactamente tras N símbolos, por lo que el relleno nunca se interpreta. También permite manejar el archivo vacío (N = 0) sin árbol.

Auditoría adicional del código base

En `build_huffman_tree`, el caso de un solo símbolo ejecuta `std::make_shared<HuffmanNode>(single, nullptr)`. Ese constructor evalúa `r->frequency`, desreferenciando un puntero nulo (comportamiento indefinido; el programa se cae con un archivo como 10^6 veces 'A'). Corrección: se agrega una hoja ficticia de frecuencia 0, de modo que el árbol es completo y el símbolo único recibe el código '0' (1 bit).

Fase 3. Benchmarking

Compilación: `g++ -O2 -std=c++17 huffman.cpp -o huffman`. Ratio de compresión = Bytes comprimidos / Bytes originales × 100 %. En todos los casos la descompresión se verificó con `cmp` (archivo idéntico al original).

Caso	Original (B)	Comprimido (B)	Ratio	H empírica (bits/byte)
Texto estructurado (≈ 1 MB, vocabulario tipo Quijote)	1,048,128	523,338	49.93 %	3.947
Binario ya comprimido (zlib, equivalente a .zip/.jpg/.mp4)	576,945	577,277	100.06 %	7.997
Datos aleatorios (1 MB, /dev/urandom)	1,000,000	1,000,332	100.03 %	≈ 8
Homogéneo: 10^6 veces 'A'	1,000,000	125,015	12.50 %	0
Archivo vacío	0	12	—	—

A. Texto estructurado

Con H ≈ 3.95 bits/byte, el límite teórico es 3.95/8 ≈ 49.3 %. Se obtuvo 49.93 %, la diferencia corresponde a la cabecera, al árbol y a que Huffman usa longitudes enteras. Nota: con el texto real del Quijote el ratio será distinto (más cercano a 55–60 %), porque el alfabeto y las frecuencias son otros; las pruebas aquí reportadas usaron texto sintético con vocabulario del Quijote.

B. Archivo previamente comprimido

Un compresor previo elimina la redundancia, así que los 256 valores de byte son casi equiprobables y $H(\mathcal{M}) \approx 8$ bits/byte. Por la desigualdad de Shannon, $L \geq H \approx 8$, de modo que Huffman no puede reducir el tamaño. Como además hay que guardar la cabecera (12 B) y un árbol de 256 hojas (≈ 320 B), el archivo crece ≈ 332 bytes (+0.03 % a +0.06 %), exactamente la sobrecarga de la Opción B.

C. Archivo homogéneo extremo

No hubo división entre cero, bucle infinito ni desbordamiento de recursión. Gracias a la hoja ficticia, el árbol tiene altura 1 y cada 'A' se codifica con 1 bit: 10^6 bits = 125,000 bytes + 15 bytes de cabecera/árbol = 125,015 bytes (12.50 %). Sin la corrección del código base, el programa se habría caído por desreferenciar un puntero nulo. La entropía es 0 bits/símbolo, pero Huffman no puede bajar

de 1 bit por símbolo; para ese caso extremo, codificación por corridas (RLE) comprimiría el archivo a unos pocos bytes.

Limitación: la implementación carga el archivo completo en memoria; para archivos de gigabytes habría que procesarlo por bloques.

Conclusiones

La codificación de Huffman alcanza prácticamente el límite de entropía en el mensaje analizado (redundancia de 0.0256 bits/símbolo), pero su beneficio real depende de empaquetar los bits (BitWriter/BitReader), de serializar el árbol con poca sobrecarga, de guardar el número de símbolos para evitar el bug del relleno y de manejar casos borde como el alfabeto de un solo símbolo. Además, no comprime datos con entropía cercana a 8 bits/byte.

Apéndice A. Código fuente completo (huffman.cpp)

```
// Compresor Huffman - ADA Práctica 3
// Uso:  ./huffman c entrada salida.huf | ./huffman d entrada.huf salida | ./huffman selftest
#include <cstdint>
#include <fstream>
#include <iostream>
#include <iterator>
#include <memory>
#include <queue>
#include <stdexcept>
#include <string>
#include <vector>

// ---------------------------------------------------------------- Nodos
struct HuffmanNode {
    uint8_t character;
    uint64_t frequency;
    std::shared_ptr<HuffmanNode> left, right;
    HuffmanNode(uint8_t ch, uint64_t f) : character(ch), frequency(f) {}
    HuffmanNode(std::shared_ptr<HuffmanNode> l, std::shared_ptr<HuffmanNode> r)
        : character(0), frequency(l->frequency + r->frequency), left(l), right(r) {}
    bool is_leaf() const { return !left && !right; }
};
using NodePtr = std::shared_ptr<HuffmanNode>;

struct NodeComparator {
    bool operator()(const NodePtr& a, const NodePtr& b) const {
        return a->frequency > b->frequency;  // montículo de mínimos
    }
};

// Corrige el bug del código base: HuffmanNode(single, nullptr) desreferenciaba nullptr.
// Con un solo símbolo se añade una hoja ficticia (freq 0) para que el árbol sea completo
// y el símbolo reciba el código "0" (1 bit).
NodePtr build_huffman_tree(const std::vector<uint64_t>& freq) {
    std::priority_queue<NodePtr, std::vector<NodePtr>, NodeComparator> heap;
    for (int i = 0; i < 256; ++i)
        if (freq[i] > 0) heap.push(std::make_shared<HuffmanNode>((uint8_t)i, freq[i]));
    if (heap.empty()) return nullptr;
    if (heap.size() == 1) {
        auto only = heap.top();
        auto dummy = std::make_shared<HuffmanNode>((uint8_t)(only->character + 1), 0);
        return std::make_shared<HuffmanNode>(only, dummy);
    }
    while (heap.size() > 1) {
        auto a = heap.top(); heap.pop();
        auto b = heap.top(); heap.pop();
        heap.push(std::make_shared<HuffmanNode>(a, b));
    }
    return heap.top();
}

// ---------------------------------------------------------------- BitWriter / BitReader
class BitWriter {
    std::vector<uint8_t> buf_;
    uint8_t cur_ = 0;
    int nbits_ = 0;  // bits ocupados en cur_
public:
    void put_bit(unsigned b) {
        cur_ = (uint8_t)((cur_ << 1) | (b & 1u));
        if (++nbits_ == 8) { buf_.push_back(cur_); cur_ = 0; nbits_ = 0; }
    }
    void put_bits(uint64_t v, int n) {  // n bits, MSB primero
        for (int i = n - 1; i >= 0; --i) put_bit((unsigned)((v >> i) & 1u));
    }
    void flush() {  // relleno con ceros a la derecha
        if (nbits_) { buf_.push_back((uint8_t)(cur_ << (8 - nbits_))); cur_ = 0; nbits_ = 0; }
    }
    const std::vector<uint8_t>& bytes() const { return buf_; }
};

class BitReader {
    const std::vector<uint8_t>& buf_;
    size_t pos_ = 0;  // posición en bits
public:
    explicit BitReader(const std::vector<uint8_t>& b) : buf_(b) {}
    unsigned get_bit() {
        if (pos_ >= buf_.size() * 8) throw std::runtime_error("fin de datos inesperado");
        unsigned b = (buf_[pos_ >> 3] >> (7 - (pos_ & 7))) & 1u;
        ++pos_;
        return b;
    }
    uint64_t get_bits(int n) {
        uint64_t v = 0;
        for (int i = 0; i < n; ++i) v = (v << 1) | get_bit();
        return v;
```

```
    }
    bool eof() const { return pos_ >= buf_.size() * 8; }
};

// ------------------------------------------------------------------ Serialización del árbol (preorden)
// nodo interno = 0 ; hoja = 1 seguido de 8 bits del carácter
void write_tree(const NodePtr& n, BitWriter& bw) {
    if (n->is_leaf()) { bw.put_bit(1); bw.put_bits(n->character, 8); }
    else { bw.put_bit(0); write_tree(n->left, bw); write_tree(n->right, bw); }
}
NodePtr read_tree(BitReader& br, int depth = 0) {
    if (depth > 300) throw std::runtime_error("árbol corrupto");
    if (br.get_bit()) return std::make_shared<HuffmanNode>((uint8_t)br.get_bits(8), 0);
    auto l = read_tree(br, depth + 1);
    auto r = read_tree(br, depth + 1);
    return std::make_shared<HuffmanNode>(l, r);
}

// Tabla de códigos (vector de bits por símbolo; la longitud puede llegar a 255)
void fill_codes(const NodePtr& n, std::vector<uint8_t>& path, std::vector<std::vector<uint8_t>>& codes) {
    if (n->is_leaf()) { codes[n->character] = path; return; }
    path.push_back(0); fill_codes(n->left, path, codes); path.pop_back();
    path.push_back(1); fill_codes(n->right, path, codes); path.pop_back();
}

// ------------------------------------------------------------------ Formato
// [ "HUF1" (4 B) ][ N = nº de caracteres originales (uint64, 64 bits) ][ árbol preorden ][ datos ][ padding ]
std::vector<uint8_t> compress(const std::vector<uint8_t>& in) {
    BitWriter bw;
    for (char c : std::string("HUF1")) bw.put_bits((uint8_t)c, 8);
    bw.put_bits(in.size(), 64);
    if (in.empty()) { bw.flush(); return bw.bytes(); }

    std::vector<uint64_t> freq(256, 0);
    for (uint8_t b : in) ++freq[b];
    auto root = build_huffman_tree(freq);
    write_tree(root, bw);

    std::vector<std::vector<uint8_t>> codes(256);
    std::vector<uint8_t> path;
    fill_codes(root, path, codes);
    for (uint8_t b : in)
        for (uint8_t bit : codes[b]) bw.put_bit(bit);
    bw.flush();
    return bw.bytes();
}

std::vector<uint8_t> decompress(const std::vector<uint8_t>& in) {
    BitReader br(in);
    std::string magic;
    for (int i = 0; i < 4; ++i) magic += (char)br.get_bits(8);
    if (magic != "HUF1") throw std::runtime_error("formato inválido");
    uint64_t n = br.get_bits(64);
    std::vector<uint8_t> out;
    if (n == 0) return out;
    out.reserve(n);
    auto root = read_tree(br);
    for (uint64_t i = 0; i < n; ++i) {  // se detiene tras N símbolos: el padding se ignora
        auto node = root;
        while (!node->is_leaf()) node = br.get_bit() ? node->right : node->left;
        out.push_back(node->character);
    }
    return out;
}

// Decodificador INGENUO (solo para demostrar el bug del padding):
// decodifica hasta agotar los bits del archivo, sin conocer N.
std::vector<uint8_t> decompress_naive(const std::vector<uint8_t>& in) {
    BitReader br(in);
    for (int i = 0; i < 4; ++i) br.get_bits(8);
    br.get_bits(64);
    auto root = read_tree(br);
    std::vector<uint8_t> out;
    try {
        while (!br.eof()) {
            auto node = root;
            while (!node->is_leaf()) node = br.get_bit() ? node->right : node->left;
            out.push_back(node->character);
        }
    } catch (...) {}  // símbolo incompleto al final: se descarta
    return out;
}

// ------------------------------------------------------------------ E/S
std::vector<uint8_t> read_file(const std::string& p) {
    std::ifstream f(p, std::ios::binary);
    if (!f) throw std::runtime_error("no se pudo abrir " + p);
    return std::vector<uint8_t>(std::istreambuf_iterator<char>(f), {});
```

```
}
void write_file(const std::string& p, const std::vector<uint8_t>& d) {
    std::ofstream f(p, std::ios::binary);
    f.write((const char*)d.data(), (std::streamsize)d.size());
}

int main(int argc, char** argv) {
    try {
        std::string mode = argc > 1 ? argv[1] : "";
        if (mode == "selftest") {
            std::string m = "ABRACADABRA_ALGORITMO_AVANZADO_ESCOM";
            std::vector<uint8_t> in(m.begin(), m.end());
            auto c = compress(in);
            auto good = decompress(c);
            auto bad = decompress_naive(c);
            std::cout << "original=" << in.size() << " B, comprimido=" << c.size() << " B\n";
            std::cout << "decoder con N : " << std::string(good.begin(), good.end()) << "\n";
            std::cout << "decoder ingenuo: " << std::string(bad.begin(), bad.end()) << "  (" << bad.size() << " chars)\n";
            return good == in ? 0 : 1;
        }
        if (argc != 4 || (mode != "c" && mode != "d")) {
            std::cerr << "uso: " << argv[0] << " c|d entrada salida | selftest\n";
            return 2;
        }
        auto in = read_file(argv[2]);
        auto out = mode == "c" ? compress(in) : decompress(in);
        write_file(argv[3], out);
        if (mode == "c")
            std::cout << "original=" << in.size() << " B  comprimido=" << out.size() << " B  ratio="
                      << (in.empty() ? 0.0 : 100.0 * out.size() / in.size()) << "%\n";
    } catch (const std::exception& e) {
        std::cerr << "error: " << e.what() << "\n";
        return 1;
    }
}
```